

BOOBUZZ current architecture

Stage 1 (Markdown) — implementation reference, checked 2026-09-16.

This document describes what is in the two code repositories now. It is not a proposal and it does not expand the contracts in `protokol.md`. The source of truth for behaviour is the code; `RobotConstants.java` is the shared compile-time configuration source, and `protokol.md` (owned by `ftc-main`) is the source of truth for the Java/Python wire seam.

The working snapshots observed while writing were:

Repository	Branch	Path	Last HEAD	commit observed
boobuzz-docs	stable	/home/shared/projects/boobuzz/docs	10fec86	
robot-code	dev-phase-1	/home/shared/projects/boobuzz/robot-code	656f166	
re-cock-nize	dev-phase-1	/home/shared/projects/boobuzz/re-cock-nize	5cc95cf	

All code and comments at these heads are English after the translation commits from `robot-cx-08` (Java, 8c23e15) and `sim-cx-04` (Python, 2a6165e). The architecture below cites only symbols present in the checked heads; older Turkish names in history are not current identifiers or new contracts.

1. Where the code lives

Repository trees

`robot-code` is the FTC Android project plus two plain-Java modules. The SDK module `FtcRobotController/` is the upstream FTC module and is outside the team's three-layer implementation.

```
robot-code/
  settings.gradle
  FtcRobotController/           # FTC SDK module
  TeamCode/
    build.gradle                # Android app; depends on :core
    core/                       # Gradle project :core (Java 17)
      build.gradle
      src/
        main/java/boobuzz/core/
          hal/                  # includes RobotConstants.java
          contract/
          logic/
            cplx_engine_1/
            controller/
            RobotLoop.java
            RobotFactory.java
          test/...
        src/main/java/org/firstinspires/ftc/teamcode/
          hal/                  # SDK-facing L1 implementation
          opmode/               # FTC shell
        sim/                   # Gradle project :sim
          build.gradle
          src/main/java/boobuzz/sim/
            SimHal.java         # TCP L1 adapter
            SimMain.java        # command-line shell
            Json.java
            *Exception.java
```

The Python repository is deliberately small at this stage:

```
re-cock-nize/
  sim/
    server.py                  # TCP protocol and lockstep
    physics.py                 # motor + mecanum + pose
    mechanism.py               # RobotConstants.java regex reader
    encoder.py                 # integer tick quantisation
    field.py                   # field constants and projection
    assets/field_biobuzz.png   # optional viewer background
    viewer.py                  # optional pygame input/drawing
```

```

tests/
|   common.py
|   test_calibrated_physics.py
|   test_determinism.py
|   test_gamepad.py
|   test_kinematics.py
|   test_mechanism.py
|   test_signal.py
|   tools/fake_client.py
# path to RobotConstants.java
# Java-free protocol client

```

Layer/package/class map

Layer or boundary	Package/module path	Key classes and data
L1 HAL contracts and configuration	boobuzz.core.hal	Hal, GamepadSource, GamepadState, RobotState, RobotAction, RobotConstants, Mechanism
L2↔L3 contract	boobuzz.core.contract	Intent, sealed Drive, Feedback, WorldSnapshot, Request, RequestStatus, RequestType
L2 common interfaces	boobuzz.core.logic	RobotEngine, Subsystem
L2 engine 1	boobuzz.core.logic.cplx_engine_1	CplxEngine1, DriveSubsystem, HalLocalizer, HalDriveTrain, PathRegistry, PedroConstants
L3	boobuzz.core.controller	Controller, GamepadController
Orchestration	boobuzz.core	RobotLoop, RobotFactory
Real L1 implementation	org.firstinspires.ftc.teamcode.hal	Hardware, RealHal (FTC SDK and Android allowed here)
FTC shell	org.firstinspires.ftc.teamcode.opmode	TeleopMain
Sim L1 adapter	boobuzz.sim in Gradle project :sim	SimHal, Json, SimMain, SimProtocolException, ServerClosedException
Python physics boundary	sim.*	SimServer, Physics, MotorSim, MecanumKinematics, Mechanism, QuantizedEncoder, Viewer
Tests	boobuzz.core.*Test, boobuzz.sim.*Test, tests/	Unit tests and socket fixtures; no production layer

2. Robot architecture

The three layers

Layer	Owns	Must not know
L1 — HAL	The clock, hardware/sim reads, motor and servo writes, gamepad source, and conversion to/from RobotState/RobotAction. RealHal owns FTC objects; SimHal owns the socket.	Drive policy, Pedro follower decisions, controller state, Python truth, or game rules. Core HAL contracts contain no hardware-map types.

L2 — Logic	Sensor interpretation, localization state, drive execution, Pedro follower/path integration, subsystem state, and conversion from an Intent to a motor/servo-level RobotAction.	FTC/Android SDK, concrete HardwareMap, Gamepad, viewer state, or simulator ground truth. It may use the SDK-free Pedro core artifact.
L3 — Controller	Policy: gamepad deadband, field-oriented conversion, and production of one Intent from one Feedback. GamepadController consumes the GamepadSource supplied by the HAL.	Concrete Real-Hal/SimHal, motor names, encoders, Pedro classes, or direct hardware writes. It cannot call the engine back.

TeamCode/core is compiled as a separate Java library. FTC/Android imports are permitted in TeamCode/hal and the OpMode shell, not in :core or :sim.

One tick

RobotLoop.tick() is single-threaded and has this exact order (the first now() call is before read() in the current implementation):

```

long now = hal.now();
RobotState state = hal.read();
Feedback feedback = engine.sense(now, state);
Intent intent = controller.decide(feedback);
RobotAction action = engine.act(intent);
hal.write(action);
ticks++;

```

CplxEngine1.sense calls every subsystem's observe(state), builds a WorldSnapshot, and drains pending request statuses. act calls every subsystem's update(intent, builder) in list order and builds one immutable action. There is no scheduler and no engine→controller reverse call. Because the engine senses before the controller decides and acts, a request submitted in this tick can only be reported by a status in a later tick. RealHal.read() obtains its own monotonic timestamp for RobotState.t; SimHal.read() returns the timestamp from the most recently received simulator state.

3. Simulator capabilities and limits

What re-cock-nize/sim currently simulates

- **One MotorSim per configured motor.** Power is clamped to $[-1, 1]$, a target RPM is $\text{power} \times \text{free_rpm} \times \text{efficiency}$, and output RPM follows it with the first-order lag $\alpha = 1 - \exp(-dt / \text{motor_tau_s})$. Integer encoder ticks are generated by QuantizedEncoder from ticks_per_rev. Non-wheel motors are also advanced and counted, although the current mechanism has only four wheels.
- **Mecanum kinematics from geometry.** For every wheel at (x_i, y_i) with roller angle γ_i , the rim-speed row is

$$v_i = (v_x - \omega y_i) - \cot(\gamma_i) * (v_y + \omega x_i)$$

The inverse is a least-squares solve of the four-row by three-column system. x is robot-forward, y is robot-left, and ω is CCW-positive. No hand-written wheel-sign table is used by the Python physics.

- **Measured lateral efficiency.** The recovered lateral velocity is multiplied by RobotConstants.STRAFE_EFF (0.7346).
- **Zero-power coasting.** When all wheel powers are zero, forward and lateral chassis velocities move toward zero by their configured decelerations. The motor outputs are adjusted to remain consistent with that chassis velocity. There is no separate calibrated angular zero-power deceleration.
- **Pose and field boundaries.** The chassis twist is transformed by heading, integrated with Euler steps, and heading is wrapped to $[-\pi, \pi]$. The field is 144×144 inches with the origin at a corner. Position is clamped to the interior using the footprint's heading-independent circumradius $\text{hypot}(\text{robot_width}, \text{robot_length}) / 2$; this is a wall boundary, not a collision solver.
- **Synthetic Pinpoint and IMU readings.** state() reports pinpoint as truth plus Gaussian noise ($\sigma_{xy} = 0.05$ in, $\sigma_h = 0.002$ rad) and imu.yaw as truth plus 0.002 rad noise. truth is included for the viewer and tests only. Java RobotState deliberately has no truth field.
- **Viewer.** viewer.py is the only pygame import. It draws a 144-inch, 24-inch-tile top view, optional BIOBUZZ background (141 in, rotated 90° CW and centred), robot footprint, trail, truth/twist/encoder/gamepad panel, and keyboard input. W/S map to ly, A/D to lx, Q/E to rx, arrow keys to dpad, SPACE to a, and ESC to quit. It does not contain physics.

- **Headless mode.** `python -m sim.server --headless` never imports `pygame` and does not impose a real-time sleep. With a viewer, the server keeps the loop near real time; without one it advances at the Java-provided timestep.
- **Determinism.** `Physics.reset(seed=...)` creates a local `random.Random`. Noise is drawn in a fixed order. The same seed and identical `step` sequence produce bit-identical state/truth sequences; changing the seed changes sensor noise, not truth. The determinism tests cover the `ready.state` plus 500 steps, reset reproducibility, and gamepad exclusion from the physics claim.

The word “electrical” needs a precise qualification. The current model carries `RobotConstants.BATTERY_V` and reports it as the bus voltage, but `MotorSim.target_rpm` does not use voltage (and there is no voltage sag, kV, or kS term). Thus the implemented per-wheel model is a first-order power-to-speed model with a fixed voltage value, not a voltage-dependent electrical circuit model.

Calibration and provenance

The canonical source is `/home/shared/projects/boobuzz/robot-code/Team-Code/core/src/main/java/boobuzz/core/hal/RobotConstants.java`. Java compiles these values into `Mechanism.DEFAULT`; Python reads the same source file with regular expressions. Values below are the current inputs; “measured” means the value was present in last season’s code/data, not that the new chassis has been measured.

Input		Current value	Origin/status
ROBOT_WIDTH, ROBOT_LENGTH		18.0, 18.0 in	Current footprint used for wall clamp/rendering
ANGLE_UNIT		"deg"	Documentation marker; motor rollerDeg values are degrees
DRIVETRAIN_TYPE		"mecanum"	Current drivetrain
WHEEL_DIAMETER		4.0 in	Current wheel geometry
BATTERY_V		12.0 V	Fixed configuration; reported, not coupled into RPM yet
MOTOR_TAU_S		0.1 s	Phase-1 model choice
EFFICIENCY_FL/FR/BL/BR		1.0 each	Default/placeholder; explicitly not measured
STRAFE_EFF		0.7346	54.09 / 73.63, carried from last season’s <code>pedroPathing/Constants.java</code>
ZERO_POWER_DECEL_FORWARD_IN_S2		36.17 in/s ²	Absolute value of last season’s <code>forwardZeroPowerAcceleration</code>
ZERO_POWER_DECEL_LATERAL_IN_S2		85.98 in/s ²	Absolute value of last season’s <code>lateralZeroPowerAcceleration</code>
each <code>RobotConstants.Motor.freeRpm</code>		351.55735379568756 rpm	Derived from 73.63 in/s and a 4-inch wheel; direct RPM was not found
each <code>RobotConstants.Motor.ticksPerRev</code>		537.7	Current motor/encoder configuration
motor positions		(±6.5, ±5.5) in	Current geometry, robot (forward, left) frame
motor rollerDeg		+45, -45, -45, +45°	Current geometry; Python converts to radians
PINPOINT offsets		x=161.0 mm, y=0.0 mm	Last season’s measured <code>Constants/Hardware</code> value; re-measure on the new chassis
SERVOS		empty array	No servos configured in this phase

The earlier sim calibration/integration run at commit 6432808 synchronised the 161/0 Pinpoint values. The current Python tests no longer carry a separate configuration fixture: `tests/common.py` points at `RobotConstants.java`, so Java and Python consume the same declarations.

What is not simulated

There is no rigid-body collision/contact solver, friction model, or `PyBullet`; only the footprint wall clamp exists. There are no field game elements, balls, scoring rules, shooter, feeder, intake, turret, ToF, Limelight, AprilTag vision, servo dynamics, sensor latency, sensor fusion, or battery sag. `step` accepts a servo map but

the Python server does not simulate it. The synthetic Pinpoint/IMU are noisy pose reports, not a model of the real sensor electronics. Pedro, controllers, paths, and robot policy run in Java, never in Python. The Java core cannot read `truth`.

4. Logic engines

`cplx_engine_N` convention

An engine iteration is a complete sibling package under `boobuzz.core.logic:cplx_engine_1`, then (when approved) `cplx_engine_2`, etc. Sibling iterations do not share implementation code; each is independently selectable and testable. The current tree contains only `cplx_engine_1`. `C1DriveEngine` was a Phase-0 remnant and has been deleted. `RobotFactory` now constructs `CplxEngine1` unconditionally, and `SimMain` has no `--engine` flag.

Contents of `cplx_engine_1`

Class	Current responsibility
<code>CplxEngine1</code>	The sole <code>RobotEngine</code> ; owns a fixed <code>List<Subsystem></code> containing the drive subsystem, senses a world snapshot, drains statuses, and assembles actions. <code>name()</code> returns " <code>cplx_engine_1</code> ".
<code>DriveSubsystem</code>	The only subsystem. It maps mechanism wheel positions to FL/FR/BL/BR, feeds the localizer, tracks <code>deltaTimeSeconds</code> , starts/stops follower commands, translates manual power, and rejects unsupported requests.
<code>HalLocalizer</code>	Pedro 3.0 Localizer bridge over <code>RobotState.pinpoint</code> ; no frame conversion. It computes velocity from successive samples, rotates velocity for a heading offset, and implements software-only <code>setPose/reset</code> .
<code>HalDrivetrain</code>	Pedro 3.0 Drivetrain bridge. It stores the last FL/FR/BL/BR output and exposes it as a <code>RobotAction</code> ; it never writes hardware.
<code>PathRegistry</code>	Registers <code>test-line</code> from <code>(72,72,0)</code> to <code>(120,72,0)</code> . <code>test-turn</code> is a hold at <code>(120,72,pi/2)</code> because Pedro 3.0 rejects a zero-length <code>Line</code> ; unknown IDs throw <code>IllegalArgumentException</code> .
<code>PedroConstants</code>	Builds a fresh <code>ForesightConfig</code> and <code>Follower</code> from Mechanism physics/geometry. PID starting values are conservative; velocity and brake parameters are derived from the <code>RobotConstants</code> values above.

Pedro 3.0 APIs actually used are `Follower(Localizer, Drivetrain, Algorithm)`, `update()/update(double)`, `follow(Path)`, `hold(Pose)` (and the available boolean overload), `is-Busy`, and `atParametricEnd`. The artifact's `DrivePowers` order is forward, strafe, turn; this is the order used by the bridge.

Drive commands that exist today

Drive variant	Units/frame	Current <code>cplx_engine_1</code> behaviour
<code>Manual(vx, omega)</code>	<code>vy</code> , Robot frame; raw power components in <code>[-1,1]</code>	Implemented. <code>DriveSubsystem</code> creates Pedro <code>DrivePowers</code> , mixes <code>FL=f-s-t</code> , <code>FR=f+s+t</code> , <code>BL=f+s-t</code> , <code>BR=f-s+t</code> , normalises the peak, and writes four wheel powers.

Velocity(vx, vy, omega)	Field frame; inches/second	Contract type only. It is not a closed-loop velocity controller yet; the subsystem falls through to <code>follower.stop()</code> , so its current output is zero. This is a future RL/Faz-6 decision, not a claimed capability.
GoTo(target, constraints)	Pose in inches/radians; constraints are <code>maxPower</code> , <code>maxVelocity</code>	Implemented through <code>follower.hold(target)</code> . The <code>constraints</code> value is carried and participates in command identity, but is not applied by this engine.
FollowPath(pathId)	Registry identifier	Implemented for test-line and test-turn through <code>PathRegistry.start</code> . Repeating the same ID does not restart the follower.
Hold()	Current pose	Implemented as <code>follower.hold(localizer.pose())</code> ; repeating <code>Hold</code> does not restart it.

For every non-manual command the subsystem calls `follower.update(deltaTimeSeconds)` after starting a changed command and copies the `HalDriveTrain`'s last action into the shared builder. The first observation has zero delta time. A transition into manual mode stops the follower before writing manual powers.

Subsystem pattern and adding one

The exact current interface is:

```
public interface Subsystem {
    void observe(RobotState state);
    void update(Intent intent, RobotAction.Builder out);
}
```

`CplxEngine1` keeps subsystems in a fixed `List` and calls `observe` in list order during `sense`, then `update` in the same order during `act`. There is no command scheduler or shared blackboard. To add a subsystem, implement these two methods in a new engine-local class, keep its state private, consume only the contract/HAL DTOs, append it at an explicit position in the engine's list, write only its named outputs to the supplied builder, and add unit tests. A future engine iteration should copy the complete engine package rather than importing an implementation from a sibling iteration.

5. Inter-module communication (the most important boundary)

Allowed call graph

```
RealHal / SimHal --Hal--> RobotLoop
RobotLoop --RobotState--> RobotEngine.sense --Feedback--> Controller.decide
RobotLoop <--Intent-- Controller
RobotLoop --Intent--> RobotEngine.act --RobotAction--> Hal.write
GamepadController --GamepadSource.get--> RealHal or SimHal
CplxEngine1 --> DriveSubsystem --> Pedro bridges (no concrete HAL)
SimHal <==== line-delimited JSON/TCP =====> re-cock-nize SimServer
```

The only mutable cross-layer values are the values passed through this tick. `RobotConstants` is compile-time configuration, not a runtime control channel. `Mechanism.DEFAULT` is one shared projection of those constants (configuration, not control state). Core has no mutable singleton scheduler or global state. `Drive.HOLD` is an immutable static value; other static values are constants or protocol defaults.

The actual mutable state is local to these objects:

Owner	State held today
RobotLoop	<code>ticks</code> counter and references to its HAL, engine, and controller.
GamepadController	Field-oriented toggle, heading offset, and previous b/y edge flags.

CplxEngine1 / DriveSubsystem	The fixed subsystem list; active drive, previous state time/delta, pending request statuses, and the Pedro/localizer/follower state.
HalLocalizer	Last raw pose/sample time, motion state, and software pose offsets.
HalDrivetrain	Four last wheel powers and configured wheel-name order.
RealHal / SimHal	Hardware/socket handles, latest state, and latest gamepad (plus sim-only truth in SimHal).
Python Physics	Per-motor output RPM/encoder state, pose, twist, simulation time, fixed voltage, and seeded RNG.
Python Viewer	Current keyboard gamepad, trail, pygame surface, and quit flag; it does not own physics.

There is no shared mutable singleton between these owners. Records cross the boundaries by value (with the map/list copying noted below), and the only cross-process shared configuration input is the canonical `RobotConstants.java` source plus the JSON messages.

L1 Java contracts

```

public interface Hal extends GamepadSource {
    long now();
    RobotState read();
    void write(RobotAction action);
}

@FunctionalInterface
public interface GamepadSource {
    GamepadState get();
}

```

`Hal.now()` is milliseconds from the HAL clock. On the simulator it is the `state.t_ms` supplied by Python; on the robot it is monotonic elapsed time from `RealHal`'s construction. `read()` is one sensor snapshot. `write()` applies one motor/servo snapshot. `Hal` extends `GamepadSource` so the controller can consume the same source without knowing which HAL implementation supplies it.

`RobotState` is exactly:

```

record RobotState(
    long t,                                // milliseconds, HAL clock
    Map<String, Integer> enc,              // motor name -> encoder ticks
    Map<String, Double> vel,              // motor name -> ticks/second
    double yaw,                           // IMU yaw, radians
    com.pedropathing.math.Pose pinpoint, // x/y inches, heading radians
    double voltage                         // volts
) {}

```

The Java record copies `enc` and `vel` with `Map.copyOf`. The `Pose` is Pedro's immutable `Pose(x(), y(), heading())`. There is deliberately no simulator truth field.

`RobotAction` is:

```

record RobotAction(
    Map<String, Double> motors,           // motor name -> power
    Map<String, Double> servos           // servo name -> position
) {}

```

Its maps are immutable. Motor values are powers in $[-1, 1]$; servo positions are the usual $[0, 1]$ range when `RealHal` writes them. A missing key reads as zero through `motor(name)/servo(name)`. `RobotAction.Builder` preserves insertion order for readable telemetry. `RealHal.write` iterates every name in `Mechanism`, clamps the value, and sends it to the FTC device. `SimHal.write` fills every configured name (missing is zero) before serialising.

`GamepadState` is the exact immutable record:

```

record GamepadState(
    double lx, double ly, double rx, double ry,
    boolean a, boolean b, boolean x, boolean y,
    boolean lb, boolean rb,
    double lt, double rt,

```

```

        Dpad dpad
    ) {
        enum Dpad { NONE, UP, DOWN, LEFT, RIGHT }
    }

```

Sticks are $[-1, 1]$; triggers are $[0, 1]$; pushing up gives a negative ly (FTC convention). `neutral()` returns all-zero/false values and `Dpad.NONE`. `RealHal.get()` maps `gamepad1`; `SimHal.get()` returns the latest protocol state `gamepad`. `GamepadController` applies a 0.05 deadband, edge-toggles field-oriented mode on `b`, records a heading offset on `y`, and returns a robot-frame `Drive.Manual`. It falls back to `yaw` if the feedback pose is null; a null gamepad source produces `Intent.idle()`.

L2/L3 contracts

The controller boundary is:

```

@FunctionalInterface
interface Controller {
    Intent decide(Feedback feedback);
}

interface RobotEngine {
    String name();
    Feedback sense(long now, RobotState state);
    RobotAction act(Intent intent);
}

```

`Intent` is the downward record:

```

record Intent(Drive drive, List<Request> newRequests, int[] cancels) {}

```

`newRequests` is copied and `cancels` is cloned (null becomes an empty array). `Intent.of(drive)` creates an intent with no requests/cancels; `Intent.idle()` uses the static `Drive.HOLD` value.

`Drive` is a sealed interface with these exact variants:

```

record Manual(double vx, double vy, double omega) implements Drive {}
record Velocity(double vx, double vy, double omega) implements Drive {}
record GoTo(Pose target, Constraints constraints) implements Drive {}
record FollowPath(String pathId) implements Drive {}
record Hold() implements Drive {}
record Constraints(double maxPower, double maxVelocity) {
    public static Constraints defaults() { return new Constraints(1.0, Double
}
static Drive HOLD = new Hold();

```

`Manual` is robot-frame normalised power. `Velocity` is field-frame inches per second. `GoTo`'s `Pose` is inches/radians. `FollowPath` carries a registry ID. `Hold` has no fields. `Constraints` currently has no effect in engine 1.

`Feedback` is the upward record:

```

record Feedback(WorldSnapshot world, List<RequestStatus> statuses, long t) {}
record WorldSnapshot(long t, Pose pose, double yaw, double voltage) {}

```

`Feedback.statuses` is copied. `CplxEngine1` fills `WorldSnapshot` from the current `RobotState` and `HalLocalizer` pose, and uses the `now` argument for `Feedback.t`. `WorldSnapshot.t` is the state timestamp; `pose` is the adjusted Pinpoint pose, `yaw` is radians, and `voltage` is volts.

Requests (currently unused; pending decision)

The declared request records are:

```

record Request(int id, RequestType type, double[] params) {}
enum RequestType { SHOOT, INTAKE }
record RequestStatus(int id, State state, double progress, String note) {
    enum State { ACCEPTED, ACTIVE, DONE, FAILED, REJECTED }
}

```

`Request.of(id, type, double... params)` and `param(index, fallback)` are the only helpers. `RequestStatus.rejected`, `.done`, and `.terminal()` are provided; there is no scheduler or mechanism handler yet. In the current engine every `Intent.newRequests` entry is rejected by `DriveSubsystem` with a status, and that status is drained on a later sense. `Intent.cancels` is not read at all. No `SHOOT` or `INTAKE` implementation exists. The request lifecycle and whether the contract should be frozen or revised are explicitly pending a team decision.

RobotConstants (compile-time configuration)

`mechanism.yaml`, `SnakeYAML`, and `MechanismLoader` have been removed. The one configuration source is `TeamCode/core/src/main/java/boobuzz/core/hal/RobotConstants.java`.

Java builds the shared `Mechanism.DEFAULT` projection from these constants, and the Python simulator reads the same Java source with regular expressions. A bad Java declaration is therefore a compile-time failure for the robot; the Python reader additionally validates the values it extracts.

The public configuration fields currently are:

Field	Type/units	Current meaning
ROBOT_WIDTH, BOT_LENGTH	double, inches	18.0 × 18.0 footprint; Python wall clamp/rendering
ANGLE_UNIT	String	"deg" marker; motor rollerDeg values are degrees
DRIVETRAIN_TYPE	String	"mecanum"
WHEEL_DIAMETER	double, inches	4.0-inch wheel
BATTERY_V	double, volts	Fixed simulator bus-voltage report (not a motor-voltage model)
MOTOR_TAU_S	double, seconds	First-order motor lag, 0.1 s
EFFICIENCY_FL, EFFICIENCY_FR, EFFICIENCY_BL, EFFICIENCY_BR	double, dimensionless	Per-wheel factors, all 1.0 today
STRAFE_EFF	double, dimensionless	0.7346 lateral-efficiency factor
ZERO_POWER_DECEL_FORWARD_IN_S2	double, inches/s ²	36.17 forward deceleration
ZERO_POWER_DECEL_LATERAL_IN_S2	double, inches/s ²	85.98 lateral deceleration
FL, FR, BL, BR	RobotConstants.Motor	Four wheel declarations, listed below
MOTORS	Motor[]	Aggregate in protocol order; Python discovers the individual declarations
SERVOS	String[]	Servo names; empty today
PINPOINT	RobotConstants.Pinpoint	Java hardware offsets/directions/type; not parsed by Python

The nested records are exact:

```

record Motor(String name, String drives, double xForward, double yLeft,
               double rollerDeg, double ticksPerRev, double freeRpm) {}
record Pinpoint(double xPodOffsetMm, double yPodOffsetMm,
                  String xPodDirection, String yPodDirection, String podType) {}

```

`RobotConstants.mechanism()` returns the one `Mechanism.DEFAULT` object. During construction, each `RobotConstants.Motor` becomes a `Mechanism.Motor` (which currently retains only `drives`, `forward`, `left`, and `freeRpm`), and `RobotConstants.PINPOINT` becomes a `Mechanism.Pinpoint`. `Mechanism` still copies its lists/maps; `wheelMotorNames()` selects `drives == "wheel"`, `motor(name)/pinpoint()` fail with `MechanismException` when absent, and `requireNames(actualMotors, actualServos)` compares sorted lists with the sim handshake.

Machine-readable line format

`sim/mechanism.py` is intentionally a small source reader, not a Java parser. The following formatting is part of the cross-repository contract:

- Numeric scalar constants must be one line in the form `public static final double NAME = NUMBER;`, with an all-capital `NAME` and a decimal/scientific numeric literal. The reader uses the `_SCALAR_RE` pattern; expressions such as `Math.PI * 4` are not machine-readable.
- String constants must be one line in the form `public static final String NAME = "value";`, again with an all-capital `NAME`. The reader uses `_STRING_RE` (currently for `DRIVETRAIN_TYPE`; it does not use `ANGLE_UNIT` to choose a conversion).
- Each motor must be a single physical line with exactly seven comma-separated constructor arguments: `public static final Motor NAME = new Motor("name", "drives", xForward, yLeft, rollerDeg, ticksPerRev, freeRpm);` `NAME` is all-capital; the first two arguments are strings and the remaining five are numeric literals. `_MOTOR_RE` does not accept a line break or an expression in the numeric positions. The four current lines are `FL`, `FR`, `BL`, and `BR`.

- `SERVOs` is read only when declared on one line as `public static final String[] SERVOS = {"servo", ...};;` an empty `{}` is valid. `MOTORS` and `PINPOINT` are Java-side declarations and are not parsed by the Python reader.

The Python `load(path)` function therefore takes a path to `RobotConstants.java`, extracts the scalar/string/motor/servo lines, converts `rollerDeg` from degrees to `roller_rad`, and validates positive/range values and four mecanum wheels. It derives per-motor efficiency from `EFFICIENCY_<NAME>` (defaulting to 1.0 if absent). `SimServer --mechanism` now names this Java source file (and defaults to the repository's `RobotConstants.java`); there is no YAML or PyYAML dependency and no separate simulator fixture. The Python `Mechanism` projection retains robot width/length, drivetrain type and diameter, roller/tick fields, battery/tau, efficiencies, and servo names; Python has no use for the Java-only Pinpoint record metadata.

Real robot and Java sim seam

`Hardware` is the SDK-only device finder/configurator. It obtains every motor and servo by the names in `Mechanism`, sets wheel direction/zero-power brake and run mode, configures the GoBILDA Pinpoint offsets/directions/pod type, sets the start pose, and collects voltage sensors. `TeleopMain` obtains `Mechanism` from `RobotConstants.mechanism()`; no runtime configuration file is loaded. `RealHal` then reads Pinpoint, encoders, velocities, IMU heading, and bus voltage and maps `gamepad1`; writes are clamped to the FTC motors/servos.

`SimHal` is the L1 TCP client. It validates the protocol version and sorted motor/servo lists against `Mechanism`, keeps the latest `RobotState` and `GamepadState`, exposes `truth()` only as a sim-test/viewer diagnostic, and never puts truth into core. `SimMain` likewise obtains the same `Mechanism.DEFAULT`; the Python-side reader (server and tests) reads the Java source path supplied by `--mechanism` (or its repository default).

JSON protocol (`SimHal` ↔ `sim.server`)

Transport is one UTF-8 JSON object per newline over TCP (default `127.0.0.1:5555`). Python is the server and physics owner; Java is the client and simulation-clock owner. Protocol version is 1.

Java → Python messages

Message	Fields and units	Semantics
<code>reset</code>	<code>type: "reset"; seed integer; pose: {x, y, h}</code> where x/y are inches and h radians	Reinitialises physics, encoders, RNG, and pose. Server defaults missing pose members and seed to zero, clamps the pose inside the field, and replies <code>ready</code> .
<code>step</code>	<code>type: "step"; positive dt_ms number (milliseconds; Java sends an integer); motors object name→power; servos object name→position</code>	Advances exactly one physics step. Motor powers are <code>[-1, 1]</code> ; missing motor keys mean zero. Java sends a complete configured name map. Python currently ignores the servo map (no servo dynamics or step-time name validation). It replies with one state.
<code>bye</code>	<code>type: "bye" only</code>	Ends this connection; there is no reply. <code>SimHal.close()</code> sends it best-effort.

Python → Java messages

`ready` has:

```
{
  "type": "ready",
  "motors": ["fl", "fr", "bl", "br"],
  "servos": [],
  "proto": 1,
  "state": {
    "type": "state",
    "t_ms": 0,
    "enc": {},
    "vel": {},
    "imu": {},
    "pinpoint": {},
    "voltage": 12.0,
    "gamepad": {},
    "truth": {}
  }
}
```

The example abbreviates maps; the actual `ready.state` has the complete state schema below. `t_ms` is zero, encoder/velocity values are zero, and the reset pose may have sensor noise. `SimHal` rejects a missing state, a nonzero initial time, a protocol version other than 1, or a motor/servo name mismatch.

Every state (including `ready.state`) contains:

Field	Type/units	Meaning
<code>type</code>	string <code>"state"</code>	Message discriminator
<code>t_ms</code>	integer milliseconds	Python simulation time; returned by <code>SimHal.now()</code>

enc	object name→integer ticks	Quantised motor encoder positions
vel	object name→number ticks/second	Delta ticks from the just-finished step divided by dt; zero for ready.state
imu.yaw	radians	Truth heading plus seeded Gaussian noise
pinpoint.x, .y	inches; .h radians	Truth pose plus seeded Gaussian noise
voltage	volts	Current fixed PhysicsConfig.battery_v value
gamepad	object below	Viewer input, or a neutral object in headless mode
truth.x, .y	inches; .h radians	Ground truth for SimHal.truth(), viewer, and tests only; never RobotState

gamepad contains lx, ly, rx, ry floats; a, b, x, y, lb, rb booleans; lt, rt floats; and lowercase dpad (none, up, down, left, or right). Java converts the dpad string to GamepadState.Dpad.

Tick semantics

The server does not advance while waiting for a step. SimHal performs a reset/ready handshake during construction. On a loop tick, read() returns the cached ready/previous state; write(action) sends step and blocks until the matching state arrives. Therefore one RobotLoop.tick() corresponds to one step/state pair and advances t_ms by exactly dt_ms. dt_ms=0 is invalid for step; the zero-duration call is used internally only to construct the initial ready.state. Viewer mode may sleep to real time; headless mode does not.

One tick on sim and on the real robot

Diagram source (Mermaid; renderer unavailable).

```
sequenceDiagram
```

```

    participant L as RobotLoop
    participant H as HAL (SimHal or RealHal)
    participant E as CplxEngine1
    participant D as DriveSubsystem/Pedro
    participant C as Controller
    participant P as Python SimServer

```

Note over H,P: SimMain uses Mechanism.DEFAULT; server --mechanism reads R

```
P-->>H: ready.state (t_ms=0)
```

```
L->>H: now()
```

```
L->>H: read()
```

```
alt simulation
```

```
    H-->>L: cached state from ready/previous step
```

```
else real robot
```

```
    H->>H: Pinpoint.update(); read encoders, IMU, voltage
```

```
    H-->>L: RobotState
```

```
end
```

```
L->>E: sense(now, state)
```

```
E->>D: observe(state)
```

```
D->>D: HalLocalizer.feed(state)
```

```
E-->>L: Feedback(WorldSnapshot, statuses, t)
```

```
L->>C: decide(feedback)
```

```
C->>H: GamepadSource.get()
```

```
H-->>C: GamepadState
```

```
C-->>L: Intent
```

```
L->>E: act(intent)
```

```
E->>D: update(intent, RobotAction.Builder)
```

```
D->>D: manual mix or Pedro follower update
```

```
E-->>L: RobotAction
```

```
L->>H: write(action)
```

```
alt simulation
```

```
    H->>P: step(dt_ms, motors, servos)
```

```
    P->>P: MotorSim, mecanum, pose, sensors
```

```
    P-->>H: state (t_ms += dt_ms)
```

```
else real robot
```

```
H->>H: setPower motors; setPosition servos
end
```

6. Build and run

Gradle layout and SDK guard

settings.gradle includes :FtcRobotController, :TeamCode, and :sim, and maps :core to TeamCode/core:

```
include ':core'
project(':core').projectDir = file('TeamCode/core')
include ':sim'
```

:core is a Java 17 java-library with Pedro core:3.0.0 and JUnit; it has no SnakeYAML or other configuration-parser dependency. :sim is Java 17 java-library + application, depends on :core, and has no TeamCode dependency. TeamCode is the Android application and depends on both FtcRobotController and :core. The wrapper is Gradle 9.1.0; the Android plugin is 8.13.2 and the FTC SDK dependencies are 12.0.0.

gradle/sdk-guard.gradle is applied to :core and :sim. Before compileJava, it scans the main and test Java source sets for com.qualcomm, org.firstinspires, or android. and fails the build on a match. A successful run writes build/sdk-guard.ok; this is a build error, not a convention that can be ignored.

Tests and APK build

```
cd /home/shared/projects/boobuzz/robot-code
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk
./gradlew :core:test :sim:test
./gradlew :TeamCode:assembleDebug
```

The first command also exercises the guard through compilation. The second builds the Android app; only the TeamCode HAL/OpMode side is allowed to use the FTC SDK.

Python tests do not require pygame:

```
cd /home/shared/projects/boobuzz/re-cock-nize
./run_tests.sh
```

Headless simulator and end-to-end run

Start the Python server first (terminal A):

```
cd /home/shared/projects/boobuzz/re-cock-nize
.venv/bin/python -m sim.server \
  --mechanism /home/shared/projects/boobuzz/robot-code/TeamCode/core/src/main
  --headless --port 5556
```

Build and run the Java client in terminal B:

```
cd /home/shared/projects/boobuzz/robot-code
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk
./gradlew :sim:installDist
sim/build/install/sim/bin/sim \
  --port 5556 \
  --path test-line --x 72 --y 72 --h 0 \
  --dt 20 --steps 1000 --seed 1
```

SimMain also accepts --drive vx,vy,omega for a fixed manual command, --host, --connect-timeout, and the usual --steps/--dt options. --path and --drive are mutually exclusive. It has no --mechanism option: the Java client always obtains Mechanism.DEFAULT through RobotConstants.mechanism(). There is no engine selector: the factory always creates cplx_engine_1.

The recorded Phase-1 integration evidence (headless, port 5556, test-line, start (72,72,0), dt=20 ms, 1,000 steps, seed=1) entered the acceptance band at step 64 and ended at approximately truth (120.0020 in, 71.9599 in, 0.000102 rad) after the Pinpoint fixture was aligned by sim commit 6432808. Two equal-seed runs were bit-identical. The real robot path has not yet been hardware-validated.

7. Appendix

Per-class reference

Class/type	Short reference
Hal	L1 clock/read/write plus GamepadSource.
GamepadSource	One GamepadState supplier.
GamepadState	Immutable sticks/buttons/triggers/dpad snapshot.

RobotState	Immutable raw sensor snapshot; no truth.
RobotAction	Immutable motor/servo output maps and builder.
RobotConstants	Compile-time Java configuration and machine-readable motor/Pinpoint records.
RobotConstants.Motor / .Pinpoint	Source records consumed by Java and (for motors/scalars) the Python regex reader.
Mechanism	Java projection of names, geometry, Pinpoint, and Pedro physics (top-level lists/maps copied); DEFAULT comes from RobotConstants .
Intent	One downward drive plus request/cancel arrays.
Drive	Sealed Manual/Velocity/GoTo/Follow-Path/Hold command family.
Feedback	World snapshot plus request statuses and time.
WorldSnapshot	Current pose, yaw, voltage, and state time.
Request, RequestType, RequestStatus Controller	Declared event lifecycle; no active mechanism handler yet. Functional Feedback → Intent boundary.
GamepadController	Deadband, field orientation, and gamepad policy.
RobotEngine Subsystem	Functional L2 sense/act boundary. Fixed-order L2 observe/update boundary.
RobotLoop	Five-stage single-thread tick.
RobotFactory	One construction path for Cplx-Engine1 + controller.
CplxEngine1	Current and only engine implementation.
DriveSubsystem	Manual and Pedro drive execution, request rejection.
HalLocalizer	Pinpoint-to-Pedro localizer bridge and software pose offset.
HalDrivetrain	Pedro output-to-RobotAction bridge; no hardware write.
PathRegistry	test-line path and test-turn hold target.
PedroConstants	Mechanism/RobotConstants -derived Foresight/Pedro configuration.
Hardware	FTC device lookup and Pinpoint/motor setup.
RealHal	FTC implementation of the core HAL.
TeleopMain	RealHal + RobotFactory Op-Mode shell.
SimHal	TCP client implementation of the core HAL.
SimMain	Java command-line runner and fixed-drive shell.
Json / sim exceptions	Line JSON codec and explicit protocol/connection errors.
Python Mechanism / Motor / PhysicsConfig	Configuration projected from RobotConstants.java .
Python MotorSim	First-order output RPM and encoder source.
Python MecanumKinematics	Geometry-derived forward/inverse chassis mapping.

Python Physics / Pose	Deterministic world state, Euler integration, clamp, noise.
Python SimServer	Lockstep TCP server and message validation.
Python Viewer / field	Optional pygame drawing/input and field projection.
Python QuantizedEncoder	Integer tick edge quantisation.

Open decisions and known gaps

- **Contract freeze.** `contract/` is the intended stable seam, but `Drive.Velocity` is not implemented, `Request/cancel` semantics are unused, and `GoTo.constraints` are not applied. `ftc-main` must decide what is frozen before later engine/subsystem phases.
- **Configuration representation (closed).** `YAML`, `SnakeYAML`, and `MechanismLoader` were removed. `RobotConstants.java` is the compile-time source because configuration errors must fail at compile time and this leaves fewer moving parts. Its one-line declaration format is now a compatibility contract for the Python regex reader.
- **Pinpoint offsets.** Current `RobotConstants.PINPOINT` and `Hardware` use measured last-season values $x=161.0$ mm, $y=0.0$ mm. The FTC sample contains an alternative $-84.0/-168.0$ mm product-insight example. Which pair belongs to the new chassis must be measured and chosen; this document does not silently change the current 161/0 configuration.
- **Duplicate configuration records.** `RobotConstants.Motor/Pinpoint` and `Mechanism.Motor/Pinpoint` currently duplicate overlapping data. Fold them into one representation in the next cleanup without breaking the Python source-reader contract.
- **Android Studio nested module sync.** `:core` is physically nested at `TeamCode/core` while being a separate Gradle project. The source/build contract is tested by Gradle, but Android Studio sync/navigation behaviour for this nested module has not been validated. The project currently requires Android Studio Narwhal 3 Feature Drop or later for AGP 8.13.2.
- **Real hardware validation.** `RealHal`, Pinpoint setup, motor directions, and `TeleOp` have not yet been exercised on the robot; `sim` is the current evidence path.
- **Translation (closed for this snapshot).** `robot-cx-08` and `sim-cx-04` completed the English rename/comment pass in commits 8c23e15 and 2a6165e; translation is not an architecture change.
- **Stage 2.** LaTeX/PDF rendering follows `ftc-main`'s Markdown review.

AI-assisted methodology

The implementation is produced and reviewed by the hierarchical workflow described in `README.md`: Tuna is the human decision-maker; `ftc-main` (Claude Fable 5.1) owns plan/protocol/specs; Codex agents implement one repository and one bounded task; zero-context Claude Opus agents review diffs. `bp` (Blueprint) is the agent-to-agent message and status channel. This document follows the same evidence rule: it records code paths, types, units, and observed results, and labels future decisions rather than presenting plans as current behaviour.